Follow-up Assignment:
Thread Control and Concurrency Models

Course: Concurrent Programming 3 (Java)

October 19, 2026

1. **Class `ThreadBall` – separation from `Ball`**

   - Create an `abstract` class `ThreadBall` that extends `Ball`.
   - Move the `static final Object MONITOR` field from `Ball` to `ThreadBall`.
   - Move the `Thread` field from `Ball` to `ThreadBall`.
   - Create a constructor that accepts parameters `x` and `y`, passing them to the superclass constructor.
   - In the constructor, recreate and start the thread, as previously done in `Ball`.
   - Override the `draw` method to set the color to black, and then call `super.draw(g)` to draw the ball.
   - move the `wakeUp` method from `Ball` to `ThreadBall`;
   - in place of `wakeUp` method create `abstract` header of this method;
   - Move the `run` method from `Ball` to `ThreadBall`.
   - In `BallPanel`, update the `actionPerformed` method to create an instance of `ThreadBall` instead of `Ball`.
   - In `BallPanel`, update the `shake` method so that the `notifyAll()` call refers to the class-level field `MONITOR` in `ThreadBall`.
   - Test that the application still works correctly, and present the results to your instructor.

2. **Class `ScheduledBall`**

   - Create an `abstract` class `ScheduledBall` that extends `Ball`.
   - Add a class-level field named `scheduler` of type `ScheduledExecutorService`, initialized with `Executors.newScheduledThreadPool(1)`.
   - Add an instance field `scheduledFuture` of type `ScheduledFuture<?>`.
   - The constructor should:
     - Accept two parameters `x` and `y`, passing them to the superclass constructor.
     - Initialize `scheduledFuture` by calling `scheduler.scheduleAtFixedRate()` with the following arguments:
       * `this` – as the task to execute,

    - `0` – as the initial delay,
    - speed – as the fixed execution interval,
    - `TimeUnit.MILLISECONDS` – as the time unit.
- Override the `draw` method to set the color to red, and then call `super.draw(g)`.
- Implement the `run` method so that:
  - If `collidingX()` returns true, multiply `vx` by `-1` and update `x`, then call `scheduledFuture.cancel(true)`;
  - If `collidingY()` returns true, multiply `vy` by `-1` and update `y`, then also call `scheduledFuture.cancel(true)`;
  - Updates coordinates `x += vx;` and `y += vy;`.
- Override the `wakeUp()` method to check whether `scheduledFuture` is canceled. If it is, reinitialize `scheduledFuture` in the same way as in the constructor.
- In `BallPanel`:
  - Extend the `actionPerformed` method so that each time it is invoked, a new `ScheduledBall` (via an anonymous inner class) is created and added to `ballList`. Pass as arguments half of the panel's width and height. Methods `collidingX()` and `collidingY()` should work the same as in `ThreadBall`.
  - Modify the `shake()` method to iterate through all elements in `ballList` and call `wakeUp()` on each.
- Test that the application still works correctly and present your results to the instructor.

3. **Classes `VBall` and `VThreadBall`**

- Create an `abstract` class `VBall` extending `Ball`.
- Add a `protected static ExecutorService vExecutor` field, initialized with `Executors.newVirtualThreadPerTaskExecutor()`.
- Implement a constructor that takes `x` and `y` and passes them to the superclass constructor.
- Create an `abstract` class `VThreadBall` extending `VBall`.
- Define the following fields:
  - `private final Lock lock = new ReentrantLock();`
  - `private final Condition condition = lock.newCondition();`
  - `private volatile boolean paused = false;`
- Create a constructor that:
  - Accepts `x` and `y`, and passes them to the superclass constructor.
  - Calls `vExecutor.submit(this)` to start the ball's virtual thread.
- Override the `draw` method to set the color to blue, and then call `super.draw(g)`.
- Create a no-argument, void method `checkPaused()` that:
  - Calls `lock.lock()`.

  - Within a `try-finally` block, loops while `paused` is true and calls `condition.await()`; finally, calls `lock.unlock()`.
- Create a void method `pause()` that:
  - Locks the `lock`, sets `paused = true`, and finally unlocks.
- Override the `wakeUp()` method to:
  - Lock the `lock`, set `paused = false`, call `condition.signalAll()`, and then unlock.
- Override the `run()` method so that in an infinite loop it:
  - Calls `checkPaused()` to handle the paused state.
  - Checks for collisions using `collidingX()` and `collidingY()`. On collision, reverses the corresponding velocity and calls `pause()`.
  - Updates coordinates `x += vx;` and `y += vy;`.
  - Sleeps for `speed` milliseconds between updates.
- In `BallPanel`, extend `actionPerformed` so that a new `VThreadBall` (via an anonymous inner class) is created each time and added to `ballList`. Pass half of the panel's width and height as arguments; methods `collidingX()` and `collidingY()` should behave as in `ThreadBall`.
- Test the application and show the results to the instructor.

4. **Class `VsThreadBall`**

- Create abstract class `VsThreadBall` extending `VBall`.
- Define a private final field `Semaphore semaphore = new Semaphore(1);`.
- Create a constructor that:
  - Accepts `x` and `y`, passing them to the superclass constructor.
  - Calls `vExecutor.submit(this)` to start the virtual thread.
- Override the `draw` method to set the color to green, and then call `super.draw(g)`.
- Override the `wakeUp()` method to call `semaphore.release()`.
- Override the `run()` method so that in an infinite loop:
  - Acquires and releases the semaphore once per iteration.
  - Checks for collisions using `collidingX()` and `collidingY()`. On collision, reverses the corresponding velocity and acquires the semaphore again.
  - Updates coordinates `x += vx;` and `y += vy;`.
  - Sleeps for `speed` milliseconds between updates.
- In `BallPanel`, extend the `actionPerformed` method to create a new `VsThreadBall` (via an anonymous inner class) each time and add it to `ballList`. Pass half of the panel's width and height as constructor arguments. Methods `collidingX()` and `collidingY()` should behave as in `ThreadBall`.
- Test that the application still works correctly, and present the results to the instructor.

5. **Final Report – Observations and Conclusions**

- Prepare a short written report summarizing your findings from this assignment.
- The report should include:
  - A brief description of how each class (`ThreadBall`, `ScheduledBall`, `VThreadBall`, `VsThreadBall`) implements concurrency differently.
  - Observations on how the program's behavior changes when switching between standard threads, scheduled executors, and virtual threads.
  - Reflections on the responsiveness and resource usage of each approach.
  - Discussion of synchronization techniques used (`MONITOR`, `Lock/Condition`, `Semaphore`) and their effect on thread control.
  - Identification of advantages and limitations of each concurrency model.
  - A short conclusion summarizing which model seems most efficient and why.
- Submit the report as `.txt` document together with your working code.
- **Do not send `.class` or `.zip` files**